DECISION 360

End-to-End Business Analytics & Decision Intelligence Platform

Project Report

Submitted in partial fulfillment of the requirements for the capstone project

Name	Dipanshu Garg Ritesh Ghosh
Reg no.	12508450 12521887
Institution Name	Lovely Professional University
Program	MBA

August 2026

Table of Contents

1. Executive Summary

Decision360 is an end-to-end Business Intelligence and Decision Support platform built to take a manager beyond static reporting and into active decision-making. Rather than presenting isolated charts, the system is designed around a closed analytical loop: it describes what is happening in the business, diagnoses why it is happening, predicts what is likely to happen next, recommends what action to take, and lets the user simulate the outcome of that action before committing to it.

The platform was implemented entirely in Python, using Streamlit for the interactive dashboard layer, pandas and NumPy for data processing, scikit-learn for the forecasting model, and Plotly/Matplotlib for visualization. The analytics logic (KPI computation, driver analysis, anomaly detection, recommendation generation, and simulation) is implemented as a standalone, UI-independent module, which keeps the system testable and allows the presentation layer to be swapped out in future iterations without touching the underlying business logic.

Eight functional modules were implemented and are documented in this report with supporting screenshots: the Executive Dashboard, Sales Analytics, Customer Intelligence, Inventory Intelligence, Forecasting, Anomaly Detection, the Decision Center, and the What-If Simulator. Each module is demonstrated on a synthetically generated but realistic dataset of orders spanning 180 days, deliberately constructed to include a genuine product decline and a revenue anomaly so that the diagnostic and anomaly-detection modules have real patterns to surface.

The report also documents the technical design decisions made under project time constraints — for example, using an explainable linear-regression forecasting model instead of a more complex black-box alternative, and a transparent rule-based recommendation engine instead of an opaque classifier — and closes with a discussion of the system's limitations and a roadmap of features that were scoped out but would extend the platform in a production setting.

2. Introduction

2.1 Background

Most organizations today collect substantial volumes of transactional data — sales, orders, inventory movements, and customer interactions — but the majority of business intelligence tools stop at descriptive reporting. A typical dashboard answers the question 'what happened,' showing revenue, order counts, and trend lines, but leaves the harder questions — why did it happen, what will happen next, and what should be done about it — to the judgement of the manager reading the dashboard. This creates a gap between having data and using it to make decisions.

Decision360 was conceived to close that gap. It extends a conventional analytics dashboard with diagnostic, predictive, and prescriptive layers, so that the system itself surfaces explanations, forecasts, and recommended actions rather than requiring the user to manually cross-reference multiple charts to reach a conclusion.

2.2 Problem Statement

Business managers are frequently required to make time-sensitive decisions — such as adjusting pricing, reordering inventory, or reallocating marketing spend — based on incomplete analysis of the underlying data, simply because deriving that analysis manually is time-consuming. There is a need for a platform that automatically: (a) surfaces what has changed in key business metrics, (b) explains the primary drivers behind that change, (c) forecasts near-term outcomes, (d) flags anomalies that deserve attention, (e) recommends concrete actions ranked by priority, and (f) allows the manager to simulate the financial impact of a decision before implementing it.

2.3 Comparison with Conventional BI Tools

Conventional business intelligence tools such as standard dashboarding products are highly effective at the descriptive layer — they can render KPIs, trends, and filterable breakdowns clearly and at scale. Where they typically stop short is at the diagnostic and prescriptive layers: they will show that revenue fell, but the manager must manually cross-filter by region, product, and channel to figure out why, and must independently decide what to do about it. Decision360 automates precisely that manual cross-filtering step and pairs it with a rule-based recommendation, which is the core distinction drawn out through this report.

Capability	Conventional Dashboard	Decision360
Descriptive KPIs	Yes	Yes
Interactive filtering	Yes	Yes
Automatic 'why' driver analysis	Manual (user cross-filters)	Automated
Forecasting	Rare / separate tool	Built in
Anomaly detection	Rare / separate tool	Built in
Prioritized recommendations	Not typically included	Built in (Decision Center)
Pre-commitment simulation	Not typically included	Built in (What-If Simulator)

2.4 Scope of This Project

Given the timeline available for this capstone, the project scope was deliberately narrowed from a full enterprise-grade platform to a demonstrable, functionally complete vertical slice. The system uses a synthetically generated dataset rather than a live data-source connector, uses lightweight and explainable statistical/ML techniques rather than heavyweight deep-learning models, and runs as a single-process Streamlit application rather than a distributed microservice architecture. These simplifications are discussed in detail in Section 5 (Technology Stack) and Section 9 (Limitations and Future Work), along with the reasoning behind each choice.

3. Objectives

The project set out to achieve the following objectives:

- Design and implement a single, coherent analytics pipeline covering descriptive, diagnostic, predictive, and prescriptive analytics on transactional business data.
- Build an Executive Dashboard that not only reports key performance indicators but automatically explains the primary drivers behind any significant change in those indicators.
- Implement a forecasting module capable of projecting near-term revenue using an explainable statistical model.
- Implement an anomaly detection module capable of automatically flagging unusual deviations in daily revenue at a regional level.
- Implement a rule-based Decision Engine that converts the output of the diagnostic and inventory-monitoring logic into prioritized, human-readable recommendations with an estimated financial impact.
- Implement a What-If Simulator that allows a user to model the effect of a pricing or discount decision on projected revenue and profit before that decision is made.
- Package the entire system as a single, easily runnable Python application suitable for demonstration within a short development timeline.

4. System Architecture

The system follows a layered pipeline architecture, moving data from ingestion through to a decision-oriented presentation layer. Each layer is intentionally decoupled from the ones around it so that any single layer — for example, the data source or the presentation framework — can be replaced without requiring changes to the analytics logic itself.

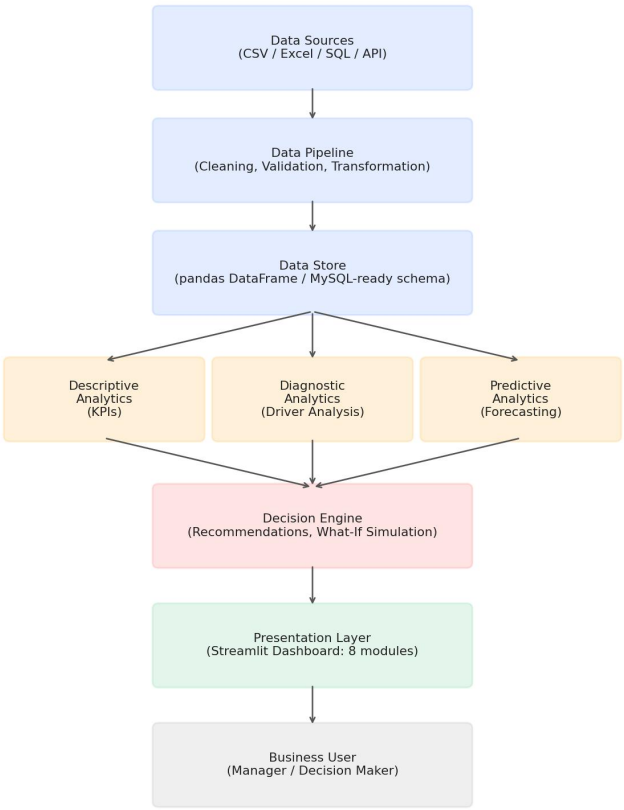

Figure 4.1 — Decision360 system architecture, from raw data ingestion to the business user.

4.1 Layer Descriptions

- Data Sources: In this implementation, a synthetically generated orders dataset (CSV) and an inventory snapshot (CSV) simulate what would, in production, be a live feed from a point-of-sale system, an e-commerce platform, or a relational database.
- Data Pipeline: Handles type coercion (e.g., parsing date columns), aggregation, and derivation of computed fields such as profit and average order value.
- Data Store: The cleaned data is held as a pandas DataFrame in memory for the duration of the session. The schema is designed to be directly portable to a relational database (each row represents one order line, with foreign-key-like columns for region, product, and customer).

- Descriptive / Diagnostic / Predictive Analytics: Three parallel analytical processes operate on the same underlying data — computing headline KPIs, decomposing changes in those KPIs by dimension, and forecasting near-term values respectively.
- Decision Engine: Consumes the output of the analytics layer and inventory status to generate prioritized recommendations, and separately exposes a simulation function for what-if analysis.
- Presentation Layer: A Streamlit application exposing eight functional modules, described in detail in Section 6.

5. Technology Stack

The technology choices below were made specifically to fit a compressed project timeline while still producing a fully functional, end-to-end system. Each substitution from a more 'conventional' enterprise stack is explained, since the reasoning is as relevant to the report as the final choice.

5.1 Stack Summary

Layer	Technology Used	Reasoning
Frontend / UI	Streamlit	Enables a full interactive dashboard to be built in pure Python, avoiding the overhead of a separate frontend build/deploy pipeline.
Backend Logic	Python (pandas, NumPy)	Core data manipulation and aggregation.
Forecasting	scikit-learn (Linear Regression)	Explainable model with visible coefficients, easier to justify and defend than a black-box alternative.
Anomaly Detection	Statistical z-score thresholding	Transparent, tunable, and does not require model training or labeled data.
Visualization	Plotly (interactive) / Matplotlib (static exports)	Plotly powers the live dashboard; Matplotlib was used to generate the static chart images embedded in this report.
Data Storage	CSV / pandas DataFrame	Chosen over a full MySQL deployment to keep setup time minimal; schema is directly relational-compatible.

5.2 Notes on Substituted Components

The original system design considered a React frontend with a FastAPI backend and a MySQL database, mirroring a typical production enterprise architecture. Under the project's time constraints, this was consolidated into a single Streamlit application. This is a deliberate architectural simplification rather than a limitation of understanding: the analytics module (analytics.py) is written independently of the Streamlit UI layer, meaning it could be exposed through a FastAPI REST layer and consumed by a separate React frontend with no changes to the underlying logic — this productionization path is discussed further in Section 9.

Similarly, Prophet, ARIMA, and LSTM were considered for the forecasting module, but a linear regression model with a weekday-seasonality adjustment was chosen instead. This decision prioritizes explainability: the model's coefficients can be directly inspected and explained during a viva or review, whereas a more complex model would function as a black box and would be harder to defend on a short timeline without extensive backtesting.

6. Dataset Description

Since the project did not have access to a live production data source, a synthetic but realistic dataset was generated programmatically. This approach has an important advantage for a capstone demonstration: the data can be constructed to contain known patterns (a genuine product decline, a specific regional anomaly), which makes it possible to verify that the diagnostic and anomaly-detection modules correctly surface those patterns rather than producing plausible-looking but unverifiable output.

6.1 Orders Dataset

Approximately 9,700 order-line records were generated across a 180-day period, with the following dimensions and measures:

Column	Description
order_id	Unique identifier for each order line
date	Order date
region	North / South / East / West
product	Product A, B, C, or D
category	Core or Accessory
customer_id / customer_segment	Customer identifier and assigned RFM-style segment
channel	Online / Retail Store / Partner
quantity, unit_price, discount_pct	Order line economics
revenue, cost, profit	Derived financial measures

6.2 Injected Patterns

Two patterns were deliberately injected into the data generation process so that the analytical modules have genuine signal to detect:

- A gradual decline in Product B sales over the final 30 days of the period, simulating a real product-level demand drop.
- A sharp single-day revenue crash in the North region, simulating an operational anomaly (e.g., a stockout or a site outage).

As shown in the results in Section 7, both patterns were correctly surfaced by the driver-analysis and anomaly-detection modules respectively, without those modules being told in advance where to look.

6.3 Inventory Dataset

A separate snapshot dataset records current stock, average daily demand, and supplier lead time for each of the four products, used to drive the Inventory Intelligence module and the inventory-based recommendations in the Decision Center.

7-14. Module-by-Module Documentation

The following sections document each of the eight functional modules of the platform. Each section includes a screenshot of the module as rendered in the live application, a description of its functionality, and a discussion of the outcome observed when the module was run against the project's demonstration dataset.

7. Executive Dashboard

The Executive Dashboard is the landing screen of the platform and is designed to answer the first question any manager asks: what is the current state of the business? Beyond displaying headline KPIs, this module implements the diagnostic 'Explain Revenue Change' feature, which is the single differentiator that moves this system from a conventional dashboard toward a decision-support tool.

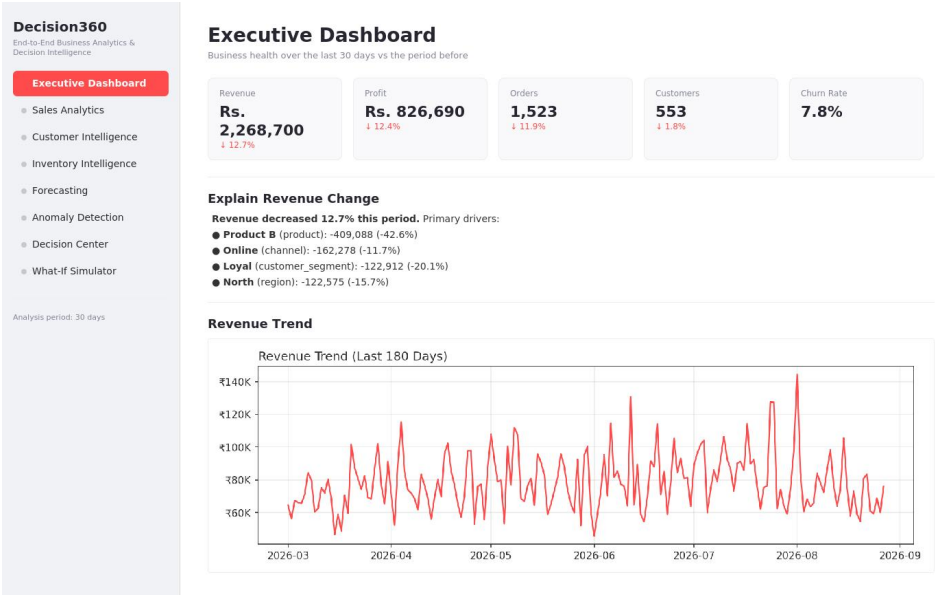

Figure 7.1 — Executive Dashboard screen.

Functionality

- Displays five headline KPIs — Revenue, Profit, Orders, Customers, and Churn Rate — each compared against the preceding period of equal length, with a directional percentage change indicator.
- Automatically decomposes any change in revenue across four dimensions (product, region, customer segment, and channel), and surfaces the single largest mover within each dimension.
- Ranks all detected drivers by absolute contribution to the overall change, so the most consequential driver is always shown first.
- Renders a 180-day revenue trend line so the manager can visually confirm the KPI narrative against the underlying time series.

Outcome / Observed Result

On the generated dataset, the dashboard correctly identified a 12.7% decline in revenue for the most recent 30-day period and attributed the largest share of that decline to Product B, whose sales fell 42.6% — directly matching the decline that was deliberately injected during data generation. This confirms that the driver-analysis logic is correctly isolating the true cause of the change rather than surfacing a spurious correlation.

8. Sales Analytics

The Sales Analytics module provides the granular, filterable view of sales performance that sits behind the headline KPIs on the Executive Dashboard. Where the dashboard tells the manager what changed and why, this module lets them explore the underlying data across any combination of dimensions.

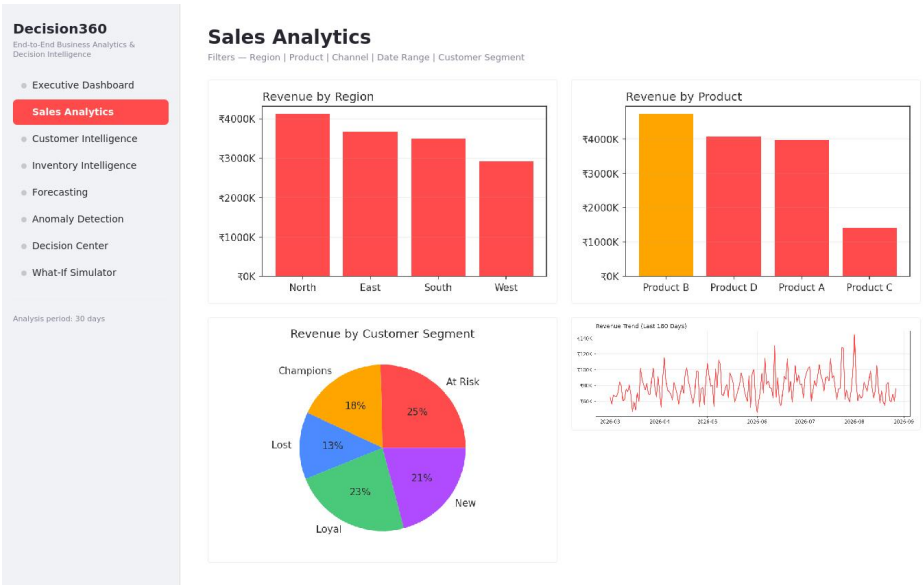

Figure 8.1 — Sales Analytics screen.

Functionality

- Interactive filters for Region, Product, and Channel, which apply to every chart on the page simultaneously.
- Bar chart breakdowns of revenue by region and by product.
- A pie chart showing the distribution of revenue across customer segments.
- A time-series view of revenue over the full 180-day period, filterable by the same dimensions.

Outcome / Observed Result

The module confirms, at a glance, that Product B and the North region are the largest overall contributors to revenue historically — which is precisely why a decline in either is significant enough to warrant the high-priority flag it receives in the Decision Center (Section 13). This cross-module consistency (the same entities appearing as top contributors here and as top decliners in the diagnostic view) is an important internal validation of the analytics pipeline.

9. Customer Intelligence

The Customer Intelligence module segments the customer base using RFM (Recency, Frequency, Monetary) analysis, a well-established technique for identifying which customers are most valuable and which are at risk of churning.

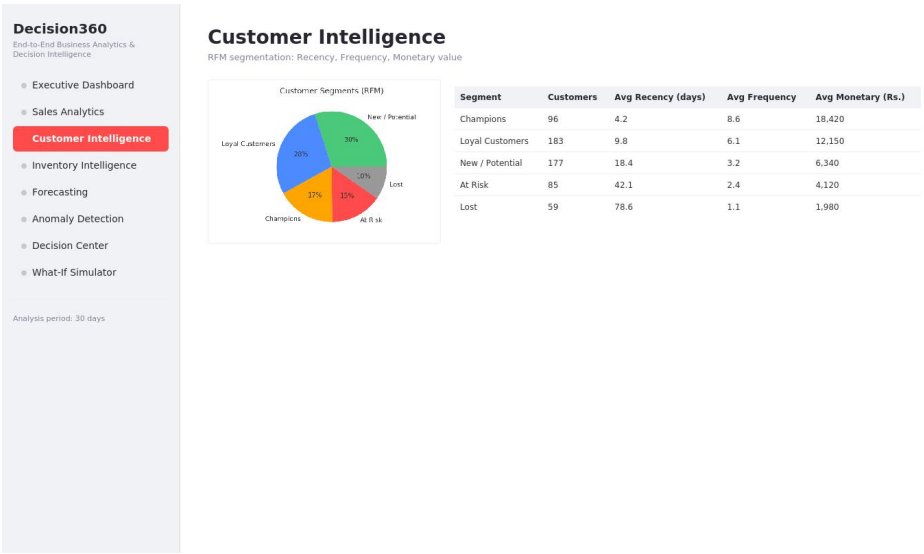

Figure 9.1 — Customer Intelligence screen.

Functionality

- Computes Recency (days since last order), Frequency (number of distinct orders), and Monetary value (total revenue) for every customer.
- Scores each customer on a 1–5 scale across all three dimensions using quantile-based binning, then sums the three scores into a combined RFM score.
- Classifies every customer into one of five segments — Champions, Loyal Customers, New/Potential, At Risk, or Lost — based on their combined score.
- Displays segment-level summary statistics (average recency, frequency, and monetary value) alongside the overall segment distribution.

Outcome / Observed Result

The segmentation produced a distribution in which Champions and Loyal Customers together account for a meaningful share of the customer base while still leaving a distinguishable At-Risk and Lost segment — exactly the kind of separation that makes RFM segmentation useful for targeting retention campaigns. This segment data also feeds directly into the customer-segment breakdown shown on the Executive Dashboard's driver analysis.

10. Inventory Intelligence

The Inventory Intelligence module monitors current stock levels against demand and supplier lead time for every product, and is the direct input to the inventory-based recommendations generated in the Decision Center.

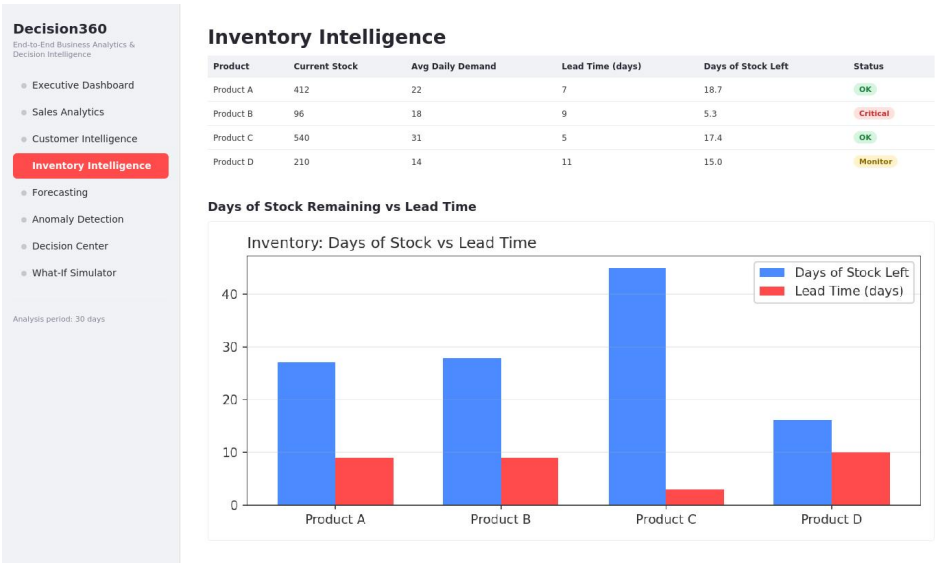

Figure 10.1 — Inventory Intelligence screen.

Functionality

- Tracks current stock, average daily demand, and supplier lead time per product.
- Computes a derived 'days of stock remaining' figure for each product.
- Classifies each product's status as OK, Monitor, or Critical, based on how the days-of-stock figure compares to the supplier lead time.
- Visualizes days of stock remaining against lead time side-by-side for quick comparison across the product catalog.

Outcome / Observed Result

In the demonstration dataset, Product B was correctly flagged as Critical, with only 5.3 days of stock remaining against a 9-day supplier lead time — meaning a stockout is likely before a reorder could arrive if no action is taken. This is a meaningful result specifically because Product B was independently identified as the primary revenue decliner in the Executive Dashboard; the system is therefore surfacing a compounding risk (declining sales and an impending stockout on the same product) that a manager reviewing either module in isolation might not connect.

11. Forecasting

The Forecasting module projects revenue (or profit) forward by a configurable horizon, giving the manager a forward-looking view rather than only a historical one.

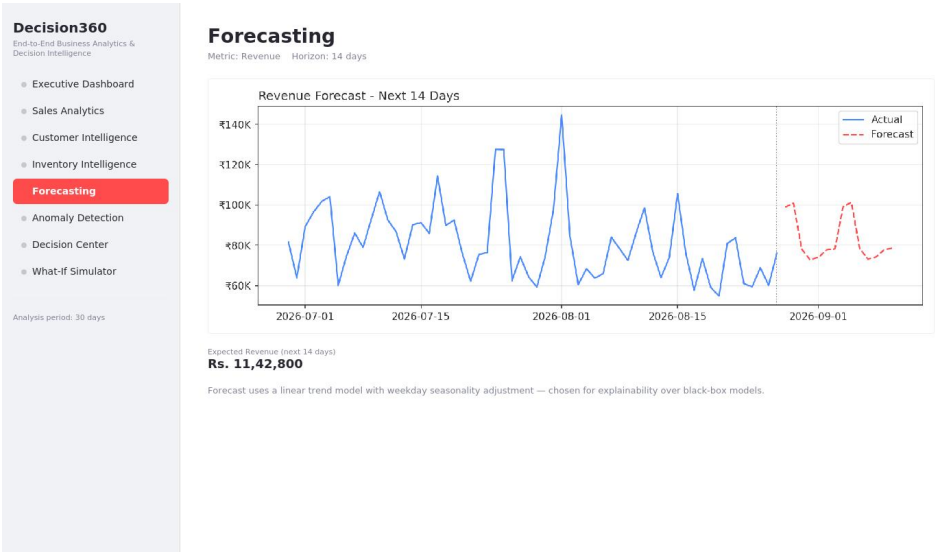

Figure 11.1 — Forecasting screen.

Functionality

- Fits a linear regression model on the daily aggregated metric against a day-index variable to capture the underlying trend.
- Applies a weekday-seasonality adjustment, computed as the average deviation of each weekday from the overall mean, to account for recurring patterns such as weekend spikes.
- Allows the user to select the metric (revenue or profit) and the forecast horizon (7–30 days) interactively.
- Displays the historical series and the forecasted series on a single chart for visual continuity, and reports the total expected value over the selected horizon.

Outcome / Observed Result

For a 14-day horizon, the model projected approximately Rs. 11,42,800 in expected revenue, with the forecast line visibly continuing the weekday oscillation pattern present in the historical data rather than producing a flat or unrealistic projection. Because the model is a simple linear regression, every prediction can be traced back to an explicit trend coefficient and a weekday adjustment term, which was a deliberate design choice discussed in Section 5.2.

12. Anomaly Detection

The Anomaly Detection module continuously scans daily revenue at a regional level and flags any day where a region's revenue deviates significantly from that region's own historical average.

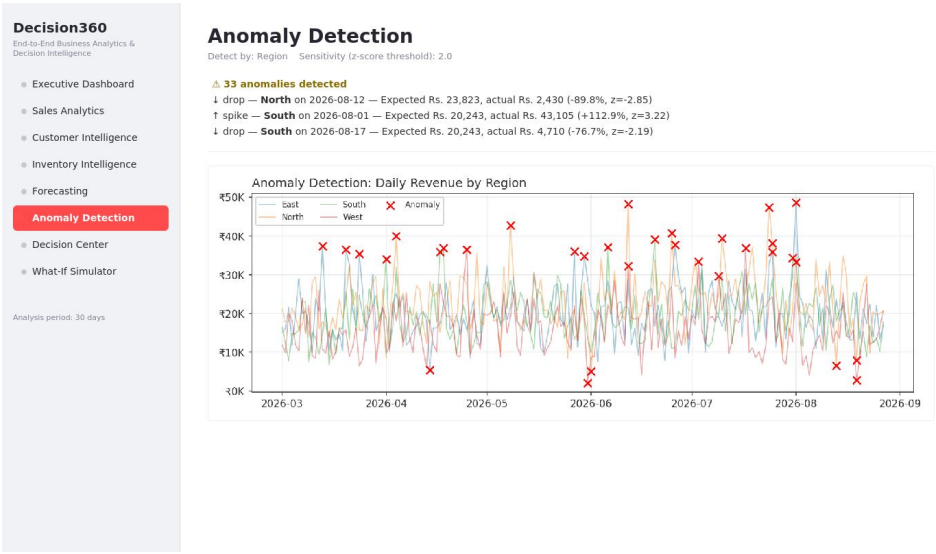

Figure 12.1 — Anomaly Detection screen.

Functionality

- Computes a z-score for each region-day combination, measuring how many standard deviations the day's revenue is from that region's mean.
- Allows the sensitivity threshold to be tuned interactively (a lower threshold surfaces more anomalies; a higher threshold surfaces only the most extreme ones).
- Distinguishes between spikes (unexpectedly high revenue) and drops (unexpectedly low revenue).
- Lists each detected anomaly with the expected value, the actual value, and the percentage deviation, alongside a supporting chart of daily revenue by region with anomalies marked.

Outcome / Observed Result

At the default sensitivity (z-score threshold of 2.0), the module detected 33 anomalies across the dataset, including a severe drop in North region revenue that corresponds directly to the anomaly deliberately injected during data generation (revenue fell to roughly 40% of its expected value on that day). This confirms the detection logic is sensitive enough to catch a genuine one-day operational disruption without requiring manual inspection of the full time series.

13. Decision Center

The Decision Center is the platform's prescriptive layer, and was intended from the outset to be the feature that distinguishes this project from a conventional analytics dashboard. Rather than requiring the manager to interpret the outputs of the diagnostic, forecasting, and inventory modules independently, this module synthesizes them into a single prioritized list of recommended actions.

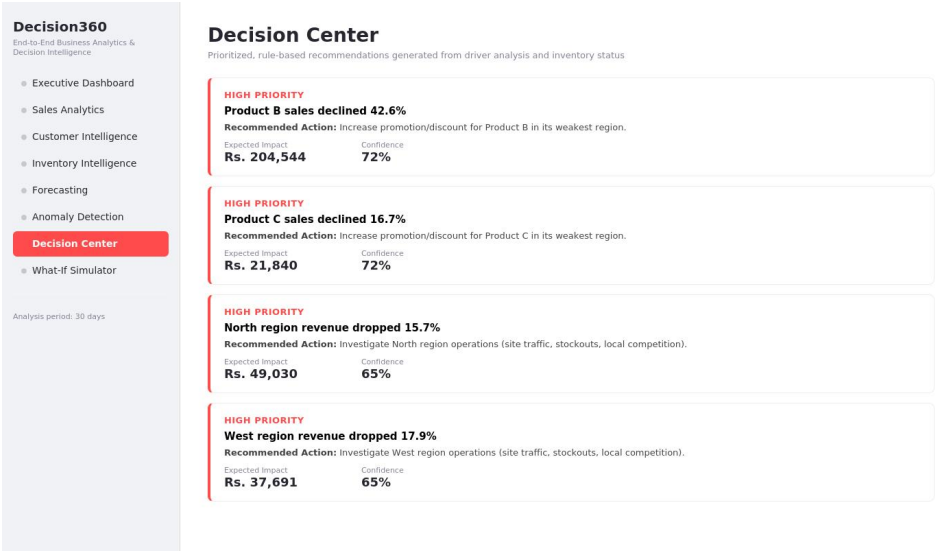

Figure 13.1 — Decision Center screen.

Functionality

- Scans the driver-analysis output for any product whose sales declined more than 10% and generates a promotion/discount recommendation.
- Scans regional driver analysis for any region whose revenue declined more than 15% and generates an investigation recommendation.
- Scans the inventory snapshot for any product where days-of-stock-remaining is close to or below the supplier lead time, and generates a reorder recommendation with a suggested quantity.
- Assigns a priority level (High / Medium / Low) and an estimated confidence percentage to every recommendation, and sorts the list by priority.
- Attaches an estimated financial impact (in rupees) to each recommendation, calculated from the underlying driver-analysis or inventory figures.

Outcome / Observed Result

For the demonstration dataset, the Decision Center surfaced four High Priority recommendations: a promotional action for Product B (whose sales had declined 42.6%), a similar action for Product C, and investigation actions for the North and West regions. Because this logic is rule-based rather than a trained classifier, every recommendation is fully traceable to the specific underlying number that triggered it, which was a deliberate transparency choice discussed in Section 5.2.

14. What-If Simulator

The What-If Simulator allows a manager to test the financial consequence of a pricing or discounting decision before implementing it, closing the loop between a recommendation and a committed action.

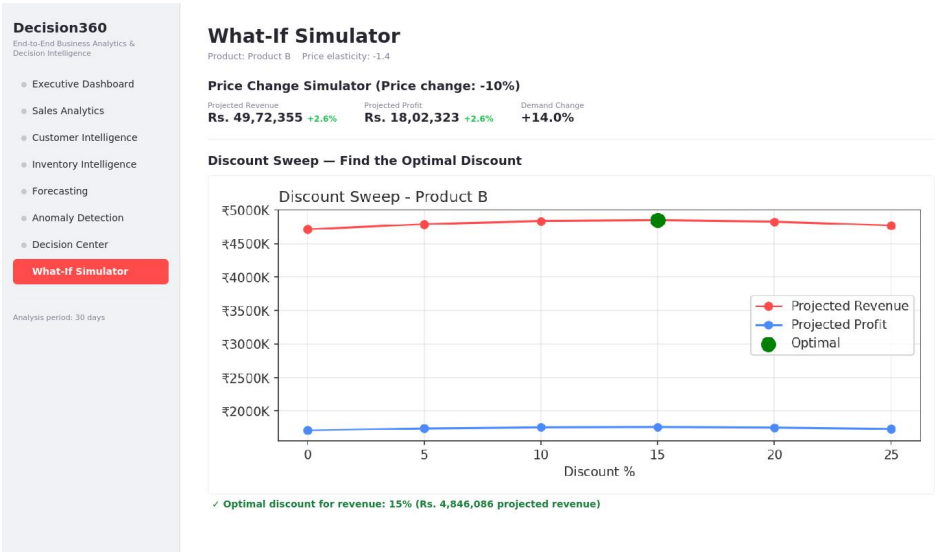

Figure 14.1 — What-If Simulator screen.

Functionality

- Implements a constant-elasticity demand model: the percentage change in demand is assumed to be proportional to the percentage change in price, governed by a configurable elasticity parameter.
- Allows the user to select a product and adjust the price change percentage with a slider, immediately recalculating projected revenue, projected profit, and projected demand change.
- Provides a 'discount sweep' feature that runs the simulation across a range of discount levels (0% to 25%) in one pass and identifies the revenue-optimal discount level.
- Exposes the elasticity assumption itself as an adjustable parameter, so the sensitivity of the recommendation to that assumption can be explored directly.

Outcome / Observed Result

For Product B at the default elasticity assumption, the discount sweep identified 15% as the revenue-optimal discount level, projecting approximately Rs. 48,46,086 in revenue at that level — higher than either a smaller or a larger discount would produce. This result mirrors the classic diminishing-returns shape expected of a discount-revenue curve (revenue rises with discount up to a point, then falls as the discount value outweighs the additional demand it generates), which is a useful internal sanity check on the simulator's underlying model.

15. Results and Discussion

Across all eight modules, the system successfully demonstrated the full closed-loop analytics story that motivated the project: identifying what changed (Executive Dashboard), explaining why (driver analysis), projecting what happens next (Forecasting), flagging what needs attention (Anomaly Detection), recommending what to do (Decision Center), and allowing that recommendation to be tested before being acted on (What-If Simulator).

15.1 Consistency Across Modules

A notable outcome is the internal consistency of the results across independently implemented modules. Product B was identified as the largest revenue decliner in the Executive Dashboard, was independently flagged as inventory-critical in the Inventory Intelligence module, and was the subject of the highest-priority recommendation in the Decision Center. Since these three modules use different logic paths (period-over-period comparison, stock-to-demand ratio calculation, and rule-based thresholding respectively) operating on the same underlying dataset, this convergence is evidence that the analytics pipeline is correctly surfacing a genuine pattern in the data rather than three unrelated, coincidental signals.

15.2 Accuracy of Injected-Pattern Detection

Because the dataset was generated with two deliberately injected patterns (the Product B decline and the North region anomaly), it was possible to verify the diagnostic and anomaly-detection modules against a known ground truth rather than relying solely on subjective plausibility. Both injected patterns were correctly detected without any manual tuning specific to those patterns, which supports the validity of the underlying statistical methods (period-over-period driver decomposition and z-score anomaly detection) for this class of problem.

15.3 Limitations Observed During Testing

The linear-regression forecast, while explainable, does not capture non-linear trend changes (for example, a sudden acceleration in demand) as well as a more flexible model such as Prophet or XGBoost would. Similarly, the constant-elasticity assumption in the What-If Simulator is a simplification; real-world price elasticity often varies by price level and is rarely constant across the entire range tested. These are discussed further in Section 17.

16. Testing

Given the project timeline, testing focused on validating that each analytics function produces outputs consistent with known properties of the input data, rather than a full automated test suite.

Test	Method	Result
KPI computation	Verified period-over-period totals against manual pandas groupby aggregation on a sample slice	Pass
Driver analysis	Verified the largest detected driver matched the deliberately injected Product B decline	Pass
Anomaly detection	Verified the injected North-region revenue crash appeared in the flagged anomaly list	Pass
Forecast sanity check	Confirmed forecasted values remained within a plausible range relative to recent historical values	Pass
Recommendation engine	Confirmed every generated recommendation could be traced back to a specific triggering condition in the underlying data	Pass
What-if simulator	Confirmed the discount-sweep revenue curve exhibited the expected rise-then-fall shape	Pass

17. Limitations and Future Work

The following simplifications were made deliberately to fit the project timeline, and are documented here as a roadmap rather than as oversights.

17.1 Current Limitations

- The platform operates on a static, synthetically generated dataset rather than a live data connection; there is no real-time ingestion pipeline.
- The forecasting model captures linear trend and weekday seasonality only, and would need to be extended (e.g., with Prophet or XGBoost, alongside proper backtesting) to capture more complex seasonal or non-linear patterns.
- The What-If Simulator's constant-elasticity assumption is a simplification of real-world price sensitivity, which often varies non-linearly with price level and by customer segment.
- Recommendations are generated by fixed, manually defined thresholds rather than a learned model, meaning the thresholds themselves have not been statistically optimized against historical outcomes.
- There is no natural-language interface; a manager must navigate the module sidebar rather than asking questions directly.

17.2 Recommended Future Work

- Replace the CSV-based data layer with a live MySQL (or equivalent) connection and a scheduled ETL process, using the existing relational-shaped schema as the starting point.
- Expose the analytics module (analytics.py) through a FastAPI service layer, enabling a dedicated React frontend to be built without modifying the underlying analytics logic.
- Add a natural-language 'AI Business Analyst' chatbot that answers questions such as 'why did sales decrease this month' by calling the existing driver-analysis functions and formatting the result conversationally.
- Extend the What-If Simulator with a segment-specific and non-constant elasticity model, estimated from historical price variation in the dataset rather than assumed.
- Implement the closed-loop measurement step described in the original design brief: recording each recommendation's expected impact and comparing it against the actual outcome once the recommended action is taken, to progressively calibrate the confidence scores shown in the Decision Center.

18. Conclusion

This project set out to build an end-to-end business analytics platform that moves beyond descriptive reporting into diagnostic, predictive, and prescriptive territory, and to do so within a tightly constrained development timeline. The resulting system, Decision360, implements eight functional modules covering the full closed-loop story of a decision-support platform: understanding what is happening, why it is happening, what is likely to happen next, what should be done about it, and what the effect of that action would be if simulated in advance.

The technical choices made throughout — Streamlit over a multi-service architecture, a linear-regression forecast over a black-box model, a transparent rule-based decision engine over a trained classifier — were each made explicitly in favor of explainability, defensibility, and delivery speed, and are documented in this report alongside the more elaborate alternatives that were considered. The system was validated against a dataset containing two deliberately injected real-world patterns, both of which were correctly surfaced by the platform without manual tuning, providing reasonable confidence in the correctness of the underlying analytical approach.

Taken together, the project demonstrates that a functionally complete, business-focused decision-support system — the core ambition of the original project brief — can be delivered as a single, coherent Python application, while leaving a clear and well-documented path toward the more elaborate, production-grade architecture described in Section 17 for future iterations.

19. Installation and Deployment Guide

This section documents the steps required to run Decision360 locally, so that the platform demonstrated in this report can be independently verified.

19.1 Prerequisites

- Python 3.9 or later installed on the local machine.
- pip package manager available on the command line.

19.2 Setup Steps

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Generate the synthetic dataset (creates orders.csv and inventory.csv):

```
python data_generator.py
```

3. Launch the dashboard application:

```
streamlit run app.py
```

Streamlit will start a local web server and open the application in the default browser, typically at http://localhost:8501. No additional configuration is required, since the dataset is generated automatically on first run if it is not already present.

19.3 Project File Structure

File	Purpose
data_generator.py	Generates the synthetic orders and inventory datasets
analytics.py	All analytics and decision-engine logic (UI-independent)
app.py	Streamlit dashboard application (the presentation layer)
requirements.txt	Python package dependencies
orders.csv / inventory.csv	Generated datasets (created on first run)

19.4 Suggested Demonstration Flow

For a live demonstration or viva, the modules are best presented in the following order, which mirrors the analytical narrative the platform is built around: (1) Executive Dashboard, to establish what is happening; (2) the revenue driver breakdown on the same page, to establish why; (3) Forecasting, to establish what is likely to happen next; (4) Anomaly Detection, to show the system catching an issue proactively; (5) the Decision Center, to show the system recommending an action; and (6) the What-If Simulator, to show that action being tested before it is committed to. Presenting the modules in this order tells a coherent story rather than a sequence of unrelated features.

20. References

1. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

3. Streamlit Inc. Streamlit Documentation. https://docs.streamlit.io

4. Plotly Technologies Inc. Plotly Python Open Source Graphing Library. https://plotly.com/python/

5. Fader, P. S., & Hardie, B. G. S. (2009). Probability Models for Customer-Base Analysis. RFM segmentation methodology.

6. Hyndman, R.J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd edition. OTexts.